

Pengujian Berorientasi Objek

By: Siti Sarah Abdullah

Tujuan Pembelajaran

- Memahami perbedaan mendasar antara pengujian perangkat lunak konvensional dan berorientasi objek (OO).
- Mengidentifikasi bagaimana konsep enkapsulasi, pewarisan (*inheritance*), dan polimorfisme memengaruhi strategi pengujian.
- Menerapkan strategi pengujian pada tingkat kelas (*class testing*) dan integrasi (*integration testing*).
- Merancang kasus uji (*test case*) yang relevan untuk sistem berorientasi objek.

Mengapa Pengujian OO berbeda?

Pergeseran paradigma

- **Konvensional:** Unit terkecil yang diuji adalah sub-program atau fungsi (modul independen)
- **Berorientasi Objek (OO):** Unit terkecil yang diuji adalah **Kelas (Class)** atau **Objek (Object)** yang mengikat data (*atribut*) dan operasi (*metode*) menjadi satu kesatuan

Tools Pengujian

- JUnit (Java)
- PyTest (Python)
- NUnit (.NET)
- Automation testing tools

Tantangan Pengujian OOP

- Kompleksitas relasi antar objek
- Efek inheritance dan polymorphism
- Reusability vs testability

Dampak Konsep OO pada Pengujian (1)

- Enkapsulasi (Encapsulation)
 - **Definisi:** Menyembunyikan detail internal objek dari dunia luar.
 - **Dampak Pengujian:** Mempersulit pengujian karena *state* internal objek (variabel *private*) tidak dapat diakses dan diamati secara langsung oleh *tester*. Kita harus mengujinya melalui metode *public* (antarmuka kelas).
- Pewarisan (Inheritance)
 - **Definisi:** Sub-kelas mewarisi sifat dari super-kelas.
 - **Dampak Pengujian:** Jika sebuah operasi telah diuji di super-kelas, apakah perlu diuji lagi di sub-kelas? **Ya.** Konteks sub-kelas berbeda, dan integrasi atribut baru bisa memicu kesalahan (*bug*) yang sebelumnya tidak muncul.

Contoh encapsulation

```
        System.out.println("--- Skenario 1: Menguji  
Donatur Biasa ---");  
        AkunDonatur donaturA = new AkunDonatur();  
        donaturA.tambahDonasi(100000); // 100rb  
(Belum VIP)  
        donaturA.potongBiayaAdmin();  
  
        // Ekspektasi: 100.000 - 5.000 = 95.000  
        System.out.println("Saldo Akhir Donatur  
Biasa: " + donaturA.getTotalDonasi());  
        System.out.println("Hasil Skenario 1:  
PASSED!\n");  
  
        System.out.println("--- Skenario 2: Menguji  
Donatur VIP (Bug Sembunyi) ---");  
        AkunDonatur donaturB = new AkunDonatur();  
        donaturB.tambahDonasi(1500000); // 1.5 Juta  
(Otomatis VIP secara internal)  
  
        // Tester memanggil operasi yang sama  
        donaturB.potongBiayaAdmin();  
  
        // Ekspektasi Tester (Karena VIP bebas  
admin): Tetap 1.500.000  
        // Realita akibat Bug di state tersembunyi:  
        1.450.000 (Dipotong 50rb)  
        System.out.println("Saldo Akhir Donatur VIP:  
" + donaturB.getTotalDonasi());  
        System.out.println("Hasil Skenario 2: FAILED!  
(Bug terjadi tanpa disadari tester)");  
    }  
}
```

```
--- Skenario 1: Menguji Donatur Biasa ---  
Saldo Akhir Donatur Biasa: 95000  
Hasil Skenario 1: PASSED!  
  
--- Skenario 2: Menguji Donatur VIP (Bug Sembunyi) ---  
Saldo Akhir Donatur VIP: 1450000  
Hasil Skenario 2: FAILED! (Bug terjadi tanpa disadari t
```

Contoh inheritance

```
    public int getTotalDitambahkan() {
        return totalDitambahkan;
    }
}

// 3. Kelas Pengujian untuk mendemonstrasikan Bug
Pewarisan
public class Main {
    public static void main(String[] args) {
        KeranjangPencatat keranjang = new
KeranjangPencatat();

        List<String> belanjaan = new ArrayList<>();
        belanjaan.add("Buku");
        belanjaan.add("Pensil");
        belanjaan.add("Penghapus");

        System.out.println("--- Masukkan 3 Barang
Sekaligus ---");
        keranjang.tambahBanyakBarang(belanjaan);

        System.out.println("Jumlah barang di memori:
" + keranjang.getJumlahBarang());
        // Hasil: 3 (Sesuai ekspektasi)

        System.out.println("Total operasi penambahan
tercatat: " + keranjang.getTotalDitambahkan());
        // EKSPEKTASI MAHASISWA: 3
        // REALITA BUG: 6 (Terjadi perhitungan
ganda!)
    }
}
```

```
--- Masukkan 3 Barang Sekaligus ---
Jumlah barang di memori: 3
Total operasi penambahan tercatat: 6
```


Dampak Konsep OO pada Pengujian (2)

- Polimorfisme (Polymorphism)
 - **Definisi:** Kemampuan objek untuk mengambil berbagai bentuk (metode yang sama bertindak berbeda pada kelas yang berbeda).
 - **Dampak Pengujian:** Meningkatkan kompleksitas. *Tester* harus memastikan bahwa pemanggilan metode dinamis (*dynamic binding*) pada saat *runtime* mengeksekusi kode yang benar untuk tipe objek yang tepat.

Contoh bug pada polymorphism

```
// 1. Super-kelas (Kelas Induk)
class PersegiPanjang {
    protected int lebar;
    protected int tinggi;

    public void setLebar(int lebar) {
        this.lebar = lebar;
    }

    public void setTinggi(int tinggi) {
        this.tinggi = tinggi;
    }

    public int getLuas() {
        return lebar * tinggi;
    }
}

// 2. Sub-kelas (Kelas Anak) yang mewarisi
// PersegiPanjang
class BujurSangkar extends PersegiPanjang {
    // Karena bujur sangkar sisinya harus sama,
    // metode di-override untuk mengubah kedua sisi
    sekaligus
    @Override
    public void setLebar(int sisi) {
        this.lebar = sisi;
        this.tinggi = sisi;
    }

    @Override
    public void setTinggi(int sisi) {
        this.lebar = sisi;
        this.tinggi = sisi;
    }
}
```

```
--- Menguji Persegi Panjang ---
Test PASSED! Luas = 20
```

```
--- Menguji Bujur Sangkar (Polimorfisme) ---
Test FAILED! Diharapkan 20, tapi hasilnya = 16
```

Strategi Pengujian OO

Pengujian perangkat lunak OO tidak dilakukan secara "top-down" tradisional, melainkan berlapis:

- **Pengujian Unit (Class Testing):** Menguji operasi-operasi di dalam kelas beserta interaksinya terhadap atribut kelas.
- **Pengujian Integrasi (Integration Testing):** Menguji kolaborasi dan komunikasi antar kelas/objek.
- **Pengujian Validasi (Validation Testing):** Menguji apakah keseluruhan sistem OO memenuhi kebutuhan pengguna (berbasis *Use Case*).

Pengujian Unit dalam OO = Pengujian Kelas

- Menguji setiap metode secara terpisah
- Menguji interaksi antar metode (bagaimana satu metode mengubah atribut yang kemudian digunakan metode lain)
- Memastikan transisi status (*state transition*) objek berjalan dengan benar dari objek diciptakan (konstruktor) hingga dihancurkan (destruktor)

Pengujian Integrasi pada Sistem OO

Karena sistem OO tidak memiliki hierarki kendali yang kaku (seperti *main program* memanggil *sub-program*), pendekatan integrasi konvensional (*top-down/bottom-up*) kurang efektif. Pendekatan Integrasi OO:

- **Thread-Based Testing:** Mengintegrasikan kumpulan kelas yang berkolaborasi untuk merespons satu input atau *event* spesifik dari sistem.
- **Use-Based Testing:** Memulai pengujian dari kelas "mandiri" (kelas server yang jarang menggunakan kelas lain), lalu perlahan mengintegrasikan kelas "dependen" (kelas klien yang menggunakan kelas server).

Desain Test Case untuk Sistem OO

- Metode desain *test case* konvensional (seperti *white-box & black-box*) tetap digunakan, namun diadaptasi:
- **Fault-Based Testing:** Merancang *test case* untuk mencari kesalahan logika atau sintaks pada operasi tertentu. (Sangat efektif untuk menguji interaksi klien-server antar objek).
- **Scenario-Based Testing:** Merancang *test case* berdasarkan *Use Case* pengguna. Menelusuri alur skenario eksekusi untuk melihat kolaborasi dinamis antar banyak objek (Sommerville menekankan pentingnya hal ini pada *system testing*).

Pengujian Validasi Berorientasi Objek

- Fokus beralih dari detail internal (*white-box*) menjadi pengujian *black-box*
- Bertujuan mengevaluasi tindakan yang terlihat oleh pengguna dan hasil (output) dari sistem
- **Alat Utama:** *Use Case* dan Diagram Aktivitas (*Activity Diagram*) yang dibuat saat fase *Requirements Engineering*. Setiap skenario *Use Case* diubah menjadi serangkaian langkah validasi.

Kesimpulan

- Sistem Berorientasi Objek menuntut penyesuaian strategi pengujian akibat karakteristik enkapsulasi, pewarisan, dan polimorfisme
- Unit dasar pengujian telah bergeser dari fungsi tunggal menjadi **Kelas**
- Pengujian integrasi di OO lebih fokus pada kolaborasi antar kelas (*thread-based* atau *use-based*) dibandingkan hierarki kontrol.
- Pengujian bergeser ke arah skenario pengguna (*Use Case*) seiring bertambah besarnya cakupan sistem

Referensi

- Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach. McGraw-Hill Education. (Konsep Strategi & Desain Test Case OO)
- Sommerville, I. (2015). Software Engineering. Pearson. (Konsep Release Testing & System Testing)
- Ammann, P., & Offutt, J. (2016). Introduction to Software Testing. Cambridge University Press. (Referensi tambahan untuk desain test case modern)
- Jorgensen, P. C. (2013). Software Testing: A Craftsman's Approach. CRC Press. (Referensi tambahan untuk pengujian berbasis model).